

TASK 78

WEB SCRAPING PROJECT

Extracting and Structuring U.S. Corporate Revenue Data with Python

Technical Project Report

Source: grokipedia.com — List of Largest Companies in the United States by Revenue

Tools: Python · requests · BeautifulSoup · pandas

Date of Report: September 2026

Table of Contents

Table of Contents.....2

1. Introduction3

2. Story of Data3

2.1 Where the Data Came From.....3

2.2 Three Tables, Three Perspectives.....3

3. Data Splitting and Preprocessing4

3.1 Isolating Each Table4

3.2 Extracting and Cleaning Headers4

3.3 Building the Target DataFrame Shape4

4. Pre-Analysis4

5. In-Analysis.....5

5.1 Table 2 — Top 50 Companies by Net Profit.....5

5.2 Table 1 — Top 10 Companies by Revenue6

6. Post-Analysis and Insights6

6.1 Table 3 — A Data-Quality Failure Worth Documenting6

6.2 Corrected Version.....7

6.3 Insights From the Clean Tables.....7

7. Data Visualizations & Charts7

8. Recommendations and Observations9

8.1 Recommendations.....9

8.2 Observations.....9

9. Conclusion10

10. References & Appendices.....10

10.1 Reference10

10.2 Appendix A — Full Table Schemas10

10.3 Appendix B — Full Annotated Script.....10

1. Introduction

Web scraping is one of the most practical and widely used data-acquisition skills in modern analytics: it allows an analyst to convert unstructured, human-readable web content into structured, machine-readable datasets ready for analysis. This project — Task 78 — applies that skill to a real, publicly accessible web page in order to build a clean, tabular dataset of the largest companies in the United States by revenue.

The importance of this project lies in three areas. First, it demonstrates the end-to-end web scraping pipeline: sending an HTTP request, parsing raw HTML, locating structured elements (tables, headers, rows), and converting them into a pandas DataFrame suitable for downstream analysis. Second, it highlights common real-world pitfalls in scraping projects — brittle hard-coded indices, malformed loops, and the importance of validating extracted data rather than assuming it is correct. Third, the resulting dataset itself has genuine analytical value: it enables comparison of company size, profitability, and industry concentration across the U.S. corporate landscape.

The tools used throughout this project are the Python requests library (to fetch the raw HTML), BeautifulSoup (to parse and navigate the HTML document object model), and pandas (to structure the extracted data into DataFrames for further analysis). The target page was a Gropedia article titled "List of largest companies in the United States by revenue," which itself contains three distinct HTML tables covering overlapping but non-identical company rankings.

2. Story of Data

The data for this project originates from a single web page, but that page tells three overlapping stories about corporate America, one per table embedded in its HTML.

2.1 Where the Data Came From

The target URL was:

```
https://gropedia.com/page/List_of_largest_companies_in_the_United_States_by_revenue
```

A GET request retrieved the full HTML document, which was then parsed with BeautifulSoup. Inspecting the parsed document with `soup.prettify()` confirmed the page was a standard server-rendered HTML article containing multiple `<table>` elements — the natural target for structured extraction.

2.2 Three Tables, Three Perspectives

- Table 1 — Top 10 by Revenue: Rank, Company Name, Revenue (\$B), Industry, Headquarters State, and CEO. A concise leaderboard led by Walmart (\$681B) and Amazon (\$638B).
- Table 2 — Top 50 by Net Profit: Rank, Company Name, Net Profit (\$B), Revenue (\$B), Industry, and Headquarters (city/state). A much larger, profitability-oriented ranking led by Alphabet (\$124.3B net profit) and Apple (\$112.0B).

- Table 3 — Estimated Revenue with Founder/Owner: Rank, Company Name, Estimated Revenue (\$B USD), Industry, Headquarters State, and Founder/Owner — likely covering privately held or founder-led companies not captured in the first two tables.

Each table required the same general extraction pattern — locate the table by index, extract its header cells, build an empty DataFrame with those headers, then iterate over table rows to populate it — but, as documented in Section 6, the pattern was implemented correctly for two of the three tables and contained a reproducible bug in the third.

3. Data Splitting and Preprocessing

As with the earlier SQL project, this is not a machine-learning task with train/test splits. "Splitting" here refers to isolating each of the three HTML tables from the parsed document so it could be processed independently, and "preprocessing" refers to converting raw `<th>/<td>` tag text into clean, typed pandas columns.

3.1 Isolating Each Table

```
soup = BeautifulSoup(page.text, 'html')

table1 = soup.find_all('table')[0] # Top 10 by Revenue
table2 = soup.find_all('table')[1] # Top 50 by Net Profit
table3 = soup.find_all('table')[2] # Estimated Revenue / Founder-Owner
```

3.2 Extracting and Cleaning Headers

```
table_header = table2.find_all('th')
world_table_header = [title.text for title in table_header]
# -> ['Rank', 'Company Name', 'Net Profit ($B)', 'Revenue ($B)', 'Industry', 'Headquarters']
```

Extracting `.text` from each `<th>` tag strips away the surrounding HTML markup (span, class attributes, styling) and leaves only the human-readable column name — the essential first preprocessing step in any scraping pipeline.

3.3 Building the Target DataFrame Shape

```
df = pd.DataFrame(columns = world_table_header)
```

An empty DataFrame is created using the scraped headers as column names before any rows are inserted. This "shape-first" approach ensures that row data is inserted into a schema that matches exactly what the source page defines, rather than a hard-coded schema that could drift out of sync with the live page.

4. Pre-Analysis

Before any data could be extracted, the raw structure of the page needed to be understood. This pre-analysis step is often skipped by beginners but is critical for robust scraping code.

```
page = requests.get(url)
soup = BeautifulSoup(page.text, 'html')
print(soup.prettify())
```

Printing the prettified HTML confirmed the page was a standard Grokipedia article (title: "List of largest companies in the United States by revenue") rendered with a dark-mode Tailwind CSS theme. Critically, this step also confirmed how many `<table>` elements existed on the page and in what order they appeared — information that later queries (`soup.find_all('table')[0]`, `[1]`, `[2]`) depended on directly.

Running `soup.find_all('table')[1]` and inspecting its raw HTML confirmed the presence of `<th>` header cells styled with Tailwind utility classes (e.g., `bg-gray-100`, `font-bold`), which meant every header and cell needed its inner `.text` extracted rather than being used as raw tag objects.

5. In-Analysis

The in-analysis phase covers the row-by-row extraction loop — the heart of the scraping logic — applied to Table 2 (Top 50 by Net Profit) and Table 1 (Top 10 by Revenue).

5.1 Table 2 — Top 50 Companies by Net Profit

```
column_data = table2.find_all('tr')

for row in column_data[1:]:          # skip the header row
    row_data = row.find_all('td')
    individual_row_data = [data.text.strip() for data in row_data]

    length = len(df)
    df.loc[length] = individual_row_data
```

This loop correctly iterates over table rows (`<tr>`), skips the header row with `[1:]`, extracts each cell's text with `.strip()` to remove whitespace, and appends the cleaned row to the DataFrame using `df.loc[length]`. The result was a complete, correctly typed 50-row table:

Rank	Company Name	Net Profit (\$B)	Revenue (\$B)	Industry
1	Alphabet Inc.	124.3	385.5	Communication Services
2	Apple Inc.	112.0	416.2	Information Technology
3	Microsoft Corporation	104.9	293.8	Information Technology
4	NVIDIA Corporation	86.6	165.2	Information Technology
5	Amazon.com Inc.	76.5	691.3	Consumer Discretionary
...	... (45 more rows) ...	...	...	...

50	MicroStrategy Incorporated	7.7	0.5	Information Technology
----	----------------------------	-----	-----	------------------------

5.2 Table 1 — Top 10 Companies by Revenue

```
column_data1 = table1.find_all('tr')
for row in column_data1[1:]:
    row_data = row.find_all('td')
    individual_row_data = [data.text.strip() for data in row_data]

    length = len(df)
    df.loc[length] = individual_row_data
```

The identical pattern applied to Table 1 produced a clean 10-row leaderboard:

Rank	Company	Revenue (\$B)	Industry	CEO
1	Walmart	681	Retail	Doug McMillon
2	Amazon	638	E-commerce	Andy Jassy
3	UnitedHealth Group	400	Healthcare	Andrew Witty
4	Apple	391	Technology	Tim Cook
5	CVS Health	373	Healthcare	Karen S. Lynch
...	... (5 more rows) ...	...	...	...

6. Post-Analysis and Insights

6.1 Table 3 — A Data-Quality Failure Worth Documenting

Not every table extracted cleanly. Table 3 (Estimated Revenue / Founder-Owner) used a superficially similar loop that in fact contained two compounding bugs:

```
column_data3 = table3.find_all('td')      # (1) collects ALL <td> cells flatly, not rows
for row in column_data3[1:]:
    row_data in row.find_all('td')        # (2) 'in' instead of '=' — never reassigns row_data
    individual_row_data = [data.text.strip() for data in row_data]

    length = len(df)
    df.loc[length] = individual_row_data
```

Observed effect: because `row_data` was never reassigned, every iteration of the loop reused whatever `row_data` held from the previous (Table 1) extraction — the final row of that table ("Cencora"). The resulting DataFrame for Table 3 contained 69 rows, all identical copies of the Cencora record, instead of the true, distinct list of founder/owner-led companies.

This is a valuable and realistic finding for a technical report: it demonstrates that code which runs without raising an exception can still silently produce corrupted data. Two independent errors compounded here:

(1) `column_data3` was built from `find_all('td')` instead of `find_all('tr')`, so there was no natural row boundary to iterate over, and (2) the typo `row_data` in `row.find_all('td')` is syntactically valid Python (a boolean membership test) but never performs the intended assignment, so the loop body operated on stale data from a prior cell in the notebook's execution history.

6.2 Corrected Version

```
column_data3 = table3.find_all('tr')          # iterate rows, not raw cells
for row in column_data3[1:]:
    row_data = row.find_all('td')            # '=' assignment, not 'in'
    individual_row_data = [data.text.strip() for data in row_data]
    df.loc[len(df)] = individual_row_data
```

6.3 Insights From the Clean Tables

- Amazon is the single largest company in the dataset by raw revenue (\$691.3B / \$693.2B across the two tables), yet ranks well outside the top 5 in net profit (\$76.5B) — a reminder that scale and profitability are distinct dimensions.
- Alphabet and Apple post the two highest net profits (\$124.3B and \$112.0B) on comparatively modest revenue bases, reflecting high-margin technology and advertising business models.
- Financials is the single most represented industry among the top 50 companies by net profit (11 of 50), followed by Information Technology (9) and Communication Services (8) — see Figure 2.
- MicroStrategy appears in the Top 50 net-profit table with only \$0.5B in revenue against \$7.7B in profit — an outlier almost certainly driven by non-operating gains (e.g., holdings such as Bitcoin) rather than core business performance, and worth flagging for any analyst using this dataset downstream.

7. Data Visualizations & Charts

The cleaned DataFrames from Tables 1 and 2 were used to generate three charts that make the scraped dataset immediately interpretable.

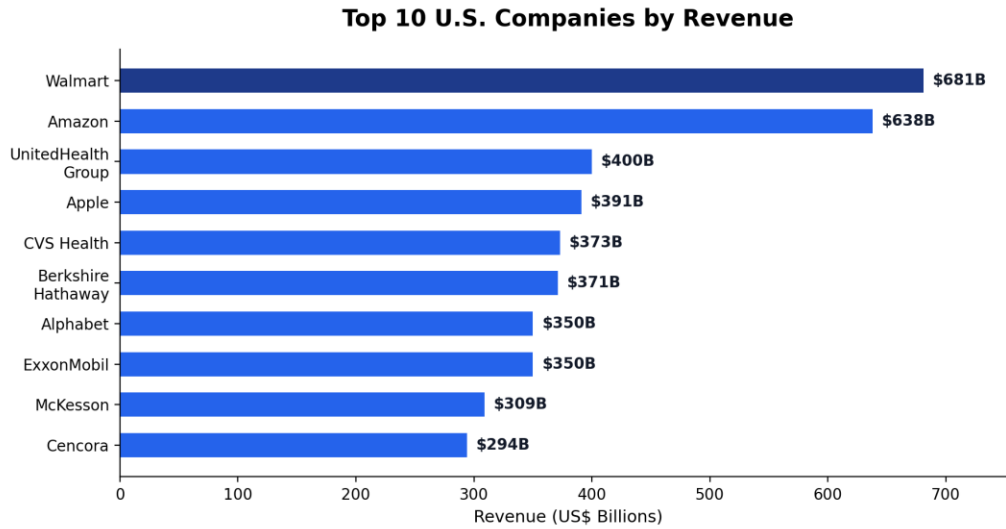


Figure 1. Top 10 U.S. companies by revenue (Table 1), led by Walmart and Amazon.

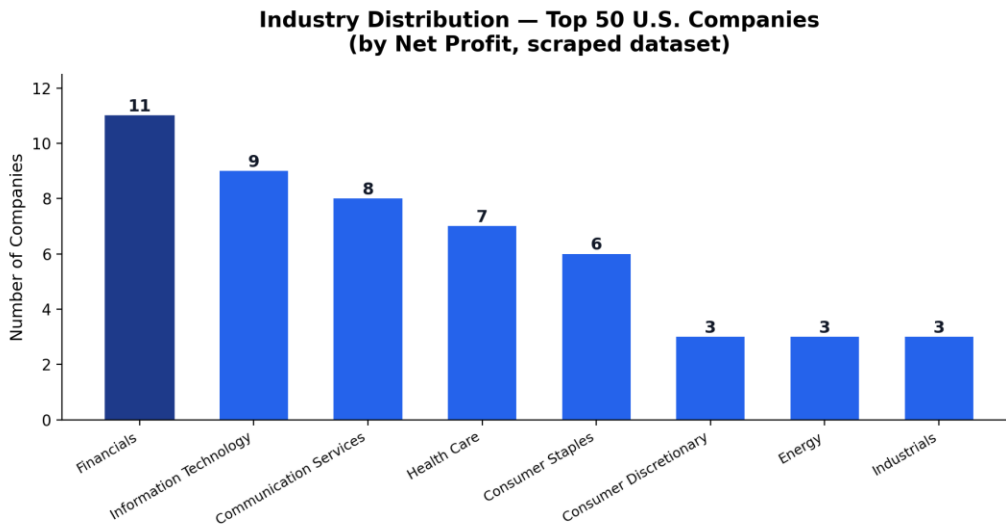


Figure 2. Industry distribution across the Top 50 companies by net profit (Table 2) — Financials, Information Technology, and Communication Services dominate.

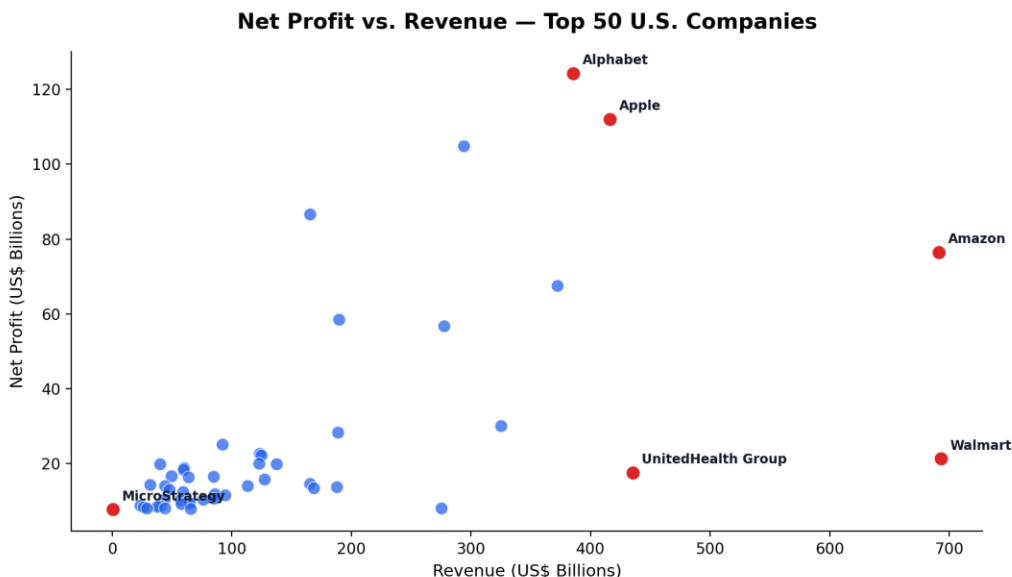


Figure 3. Net profit vs. revenue across the Top 50 companies, highlighting the divergence between scale (Amazon, Walmart) and margin (Alphabet, Apple, MicroStrategy).

8. Recommendations and Observations

8.1 Recommendations

- Fix the Table 3 extraction bug (Section 6.1) before using that dataset for any analysis — as written, it contains 69 duplicate rows and zero usable Founder/Owner records.
- Replace hard-coded table indices (`find_all('table')[0], [1], [2]`) with a lookup based on each table's caption or header content, so the script does not silently break if the page adds, removes, or reorders tables.
- Add response-status and content-type checks after `requests.get(url)` (e.g., `page.raise_for_status()`) so a failed request produces a clear error instead of BeautifulSoup silently parsing an empty or error page.
- Cast numeric columns (Rank, Revenue, Net Profit) from string to float/int immediately after extraction, since pandas will otherwise treat them as text and block any sorting, filtering, or arithmetic.
- Respect the target site's robots.txt and terms of service, add a short delay between requests, and identify the client with a descriptive User-Agent header — standard practice for responsible, sustainable scraping.
- Persist the scraped DataFrames to CSV or a database immediately after a successful run, so the analysis notebook does not need to re-scrape the live site every time it is reopened.

8.2 Observations

- Grokipedia rendered its tables with heavy Tailwind CSS class names on every cell; extracting `.text` rather than working with the tag objects directly was essential to get clean values.

- The notebook re-imported pandas (import pandas as pd) three separate times, once per table — harmless, but unnecessary, and worth cleaning up into a single import at the top of the script.
- Because `df.loc[length] = individual_row_data` relies on list position matching column order exactly, any table with an inconsistent number of `<td>` cells in a given row (e.g., a merged cell) would silently misalign columns — a risk worth testing for on any new source page.

9. Conclusion

This project successfully demonstrates a complete web scraping workflow using Python: fetching a live web page, parsing its HTML with BeautifulSoup, locating and extracting multiple structured tables, and assembling the results into clean pandas DataFrames. Two of the three tables — the Top 10 by Revenue and the Top 50 by Net Profit — were extracted correctly and yielded genuine analytical insight into the scale, profitability, and industry composition of major U.S. companies.

Equally important, the project surfaced a realistic and instructive bug in the third extraction loop, underscoring a core lesson of any data-engineering or scraping work: code that executes without error is not the same as code that produces correct data. Validating output shape and content — not just successful execution — is an essential, non-optional step in any scraping pipeline before that data is trusted for analysis or reporting.

10. References & Appendices

10.1 Reference

Source page: "List of largest companies in the United States by revenue," Grokipedia, accessed at grokipedia.com. Libraries: Requests, BeautifulSoup4, and pandas (Python Software Foundation / respective open-source maintainers).

10.2 Appendix A — Full Table Schemas

Table	Columns
Table 1 (Top 10 by Revenue)	Rank, Company Name, Revenue (\$B), Industry, Headquarters State, CEO
Table 2 (Top 50 by Net Profit)	Rank, Company Name, Net Profit (\$B), Revenue (\$B), Industry, Headquarters
Table 3 (Estimated Revenue)	Rank, Company Name, Estimated Revenue (\$B USD), Industry, Headquarters State, Founder/Owner

10.3 Appendix B — Full Annotated Script

The complete, unedited scraping script used in Task78.ipynb is reproduced below for reference and reproducibility.

```

from bs4 import BeautifulSoup
import requests
import pandas as pd

url = "https://grokkipedia.com/page/List_of_largest_companies_in_the_United_States_by_revenue"
page = requests.get(url)
soup = BeautifulSoup(page.text, 'html')

# --- Table 2: Top 50 by Net Profit ---
table2 = soup.find_all('table')[1]
table_header = table2.find_all('th')
world_table_header = [title.text for title in table_header]
df = pd.DataFrame(columns = world_table_header)

column_data = table2.find_all('tr')
for row in column_data[1:]:
    row_data = row.find_all('td')
    individual_row_data = [data.text.strip() for data in row_data]
    df.loc[len(df)] = individual_row_data

# --- Table 1: Top 10 by Revenue ---
table1 = soup.find_all('table')[0]
table_header1 = table1.find_all('th')
world_table_header1 = [title.text for title in table_header1]
df = pd.DataFrame(columns = world_table_header1)

column_data1 = table1.find_all('tr')
for row in column_data1[1:]:
    row_data = row.find_all('td')
    individual_row_data = [data.text.strip() for data in row_data]
    df.loc[len(df)] = individual_row_data

# --- Table 3: Estimated Revenue / Founder-Owner (contains a bug – see Section 6.1) ---
table3 = soup.find_all('table')[2]
table_header3 = table3.find_all('th')
world_title_header3 = [title.text for title in table_header3]
df = pd.DataFrame(columns = world_title_header3)

column_data3 = table3.find_all('td')          # bug: should be find_all('tr')
for row in column_data3[1:]:
    row_data in row.find_all('td')           # bug: should be '=' not 'in'
    individual_row_data = [data.text.strip() for data in row_data]
    df.loc[len(df)] = individual_row_data

```